

Running Claude Code in Docker on Ubuntu

Start to finish — installing Docker Engine, building the image, and authenticating reliably

Revision note: this version folds in fixes for the two problems that actually bite in practice — a root-owned `.claude` directory silently breaking login, and ephemeral containers losing auth between sessions. Both are addressed directly in the Dockerfile and run command below, not left as follow-up troubleshooting.

This covers a bare Ubuntu machine with nothing installed yet. The sequence is:

1. Install Docker Engine on the Ubuntu host
2. Set it up to run without `sudo`
3. Build an image with Claude Code in it — with ownership handled correctly up front
4. Run **one persistent, named** container and sign in
5. Reconnect to that same container every time afterward
6. Optional — start that container at a shell instead of Claude, to install more into it first

Docker has to go on the host first — the Dockerfile is only a recipe, and `docker build` is what executes it.

1. Install Docker Engine

Use Docker's own apt repository rather than Ubuntu's `docker.io` package. The distro package lags behind and doesn't include Compose v2 or buildx.

Supported on Ubuntu 22.04, 24.04, and 26.04 LTS (64-bit).

```
# Remove any conflicting older packages
sudo apt remove docker.io docker-compose docker-doc podman-docker containerd runc

# Add Docker's signed apt repo
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

# Install
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin

# Verify
sudo docker run hello-world
```

The `apt remove` in the first step will complain about packages you don't have. That's harmless.

Docker should already be enabled on boot after install; confirm with `sudo systemctl enable --now docker`.

2. Run Docker without sudo

Otherwise every `docker build` and `docker run` needs elevation:

```
sudo usermod -aG docker $USER
newgrp docker # or just log out and back in
docker run hello-world
```

Know what you're trading. The `docker` group is effectively root. Anyone in it can mount the host filesystem into a privileged container and take over the machine. On your own dev box that's a normal call to make; on a shared or production host, think twice.

`newgrp docker` only fixes the shell you run it in. Log out and back in so every new terminal picks up the group.

3. Build the Claude Code image

Anthropic publishes a signed apt repository, which is the most reproducible option for containers. Packages installed this way do not auto-update, so your image stays pinned to whatever it was built with.

What this step actually is

Nothing runs in step 3. You are writing one plain-text file — a recipe. Docker doesn't read it until step 4, when `docker build` executes it line by line.

Three details make the step work:

- The filename must be exactly `Dockerfile` — capital D, no extension. Not `dockerfile`, not `Dockerfile.txt`. If `docker build` later says `failed to read dockerfile`, this is why.
- Put it in its own empty directory. Docker uploads the entire directory to the build daemon as the “build context.” Run it in your home directory and you'll ship gigabytes to the daemon before the build even starts.
- The `.` in `docker build -t claude-dev .` means “the `Dockerfile` and the build context are here, in the current directory.” That is the link between step 3 and step 4.

Create the directory and the file

```
mkdir -p ~/claude-docker
cd ~/claude-docker
nano Dockerfile
```

Paste the content below into the editor, then press `Ctrl+O`, `Enter` to write it, and `Ctrl+X` to exit. Confirm it landed correctly with `ls -l` — you should see one file named `Dockerfile` and nothing else.

```
FROM ubuntu:24.04
ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y --no-install-recommends \
    ca-certificates curl gnupg git ripgrep sudo \
    && rm -rf /var/lib/apt/lists/*
```

```

# Claude Code signed apt repo
RUN install -d -m 0755 /etc/apt/keyrings \
  && curl -fsSL https://downloads.claude.ai/keys/claude-code.asc \
    -o /etc/apt/keyrings/claude-code.asc \
  && gpg --show-keys --with-colons /etc/apt/keyrings/claude-code.asc \
    | grep -q '^fpr:*31DDDE24DDFAB679F42D7BD2BAA929FF1A7ECACE:' \
  && echo "deb [signed-by=/etc/apt/keyrings/claude-code.asc] \
    https://downloads.claude.ai/claude-code/apt/stable stable main" \
    > /etc/apt/sources.list.d/claude-code.list \
  && apt-get update && apt-get install -y claude-code \
  && rm -rf /var/lib/apt/lists/*

# Run as non-root -- the CLI refuses --dangerously-skip-permissions as root
RUN useradd -m -s /bin/bash dev \
  && echo "dev ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/dev

# --- Auth persistence fix ---
# Pre-create the config directory and hand ownership to "dev" *before*
# switching users. If this is skipped, some install/first-run paths leave
# /home/dev/.claude owned by root: login then reports "successful" but the
# CLI can't write the credential file into a directory it doesn't own, and
# the next check silently reports "not logged in".
RUN mkdir -p /home/dev/.claude && chown -R dev:dev /home/dev/.claude

USER dev
WORKDIR /workspace
CMD ["claude"]

```

What each line does

Line	Purpose
FROM ubuntu:24.04	The starting point — a bare Ubuntu filesystem pulled from Docker Hub. Every line below modifies it.
ENV DEBIAN_FRONTEND=noninteractive	Stops apt from asking questions during the build. Nobody is at the keyboard, so a prompt would hang forever.
RUN apt-get update && apt-get install...	Installs the supporting tools. The trailing <code>rm -rf /var/lib/apt/lists/*</code> deletes the package index again so it doesn't bloat the image.
install -d -m 0755 /etc/apt/keyrings	Creates the directory apt expects signing keys to live in.
curl ... claude-code.asc	Downloads Anthropic's package signing key.
gpg --show-keys ... grep -q '^fpr:*31DD...'	Integrity check. If the downloaded key isn't Anthropic's, grep exits non-zero and the build stops right here rather than installing something unverified.
echo "deb [signed-by=...]" > ...list	Registers the Claude Code apt repository, on the stable channel.
apt-get install -y claude-code	The actual install. Everything above exists to make this line safe.
useradd -m -s /bin/bash dev	Creates an ordinary user with a home directory and a login shell.

Line	Purpose
<code>mkdir -p /home/dev/.claude && chown -R dev:dev ...</code>	New. Pre-creates the config directory owned by dev, before USER dev switches identity — this is the fix for the silent-login-failure bug.
<code>USER dev</code>	Switches identity. Every instruction after this — and the container itself — runs as dev, not root.
<code>WORKDIR /workspace</code>	The directory Claude starts in. Step 4 mounts your host project folder onto it.
<code>CMD ["claude"]</code>	What runs when you docker run the image without naming a command.

Reading the syntax

A backslash at the end of a line is a line continuation — Docker joins it to the next line before running anything. `&&` chains shell commands so each only runs if the previous one succeeded. Each RUN instruction becomes one image layer, which is why many commands are bundled into a few long RUNs instead of a dozen short ones: fewer layers, smaller image. If any command in a chain fails, the whole build fails at that layer.

Watch out for copy-paste line breaks. If you copy the Dockerfile out of a PDF or a web page, a wrapped line can arrive as two separate lines with the backslash lost. The build then fails with a confusing shell error. After pasting, check that every continued line still ends in `\`.

The fingerprint

The `grep` on the fingerprint is a build-time integrity check — it fails the build if the downloaded key isn't Anthropic's. The expected value is:

```
31DD DE24 DDFA B679 F42D 7BD2 BAA9 29FF 1A7E CACE
```

Anthropic's documentation prints it with spaces for readability; the `grep` pattern above uses the unspaced form because that's what `gpg --with-colons` emits. If `apt update` ever fails with `NO_PUBKEY BAA929FF1A7ECACE`, the key download is what went wrong.

Alternatives to the apt repo

- **Native installer:** `curl -fsSL https://claude.ai/install.sh | bash` — lands in `~/.local/bin`. Run it as the non-root user. It self-updates in the background unless you set `DISABLE_AUTOUPDATER=1`.
- **npm:** `npm install -g @anthropic-ai/claude-code` if the image already has Node 22+. Never use `sudo npm install -g`.

If step 3 goes wrong

Symptom	Cause
failed to read dockerfile: no such file or directory	The file isn't named Dockerfile, or you're not in the directory that holds it. Check with <code>ls -l</code> .
Build hangs at "transferring context"	The Dockerfile isn't in an empty directory — Docker is uploading everything around it.

Symptom	Cause
Build stops at the grep line	The key didn't download, or the fingerprint doesn't match. Check outbound access to downloads.claude.ai from the build host.
/bin/sh: syntax error: unexpected end of file	A continuation backslash was lost when pasting. Re-check the line endings.

4. Build and run — one named container, once

This is the step that actually determines whether login sticks. The single most common way this breaks is running `docker run claude-dev` repeatedly: every bare `docker run` without `--name` creates a brand-new container with a random name (`silly_liskov`, `exciting_shaw`, and so on). Each one starts with a blank filesystem, so a login done in one has nothing to do with the next. It looks like “it logged out on its own” when it’s actually a different container each time.

Build the image

```
docker build -t claude-dev .
```

Create the container — once, with a name

Mount a named volume so you don’t have to re-authenticate every time the container is recreated, and give the container itself a fixed name so you always know which one to reconnect to:

```
docker run -it --name claude-dev-main \
  -v "$PWD":/workspace \
  -v claude-config:/home/dev/.claude \
  -e CLAUDE_CONFIG_DIR=/home/dev/.claude \
  claude-dev
```

Note the change from the original recipe: this drops `--rm` and adds `--name claude-dev-main`. `--rm` deletes the container the moment it exits — combined with forgetting to reconnect to a named container, that’s the other way people lose auth. The named volume (`claude-config`) already makes credentials durable even across a container rebuild; the container itself doesn’t also need to be disposable.

`CLAUDE_CONFIG_DIR` matters. Claude Code normally writes `~/.claude.json` outside the `~/.claude` directory, so mounting the volume alone won’t keep you signed in. Pointing the config dir at the volume puts both inside it.

Every time after that: reconnect, don’t recreate

```
# If it's stopped:
docker start -ai claude-dev-main

# If it's already running, attach a new shell to it:
docker exec -it claude-dev-main bash
```

Closing a terminal window does not stop the container by itself; it just detaches you. But if your habit is to open a new terminal and type `docker run claude-dev` again out of muscle memory, you’ll get a new, unauthenticated container every time regardless of what’s mounted. `docker ps -a` is the way to check what actually exists before assuming something was lost:

```
docker ps -a
```

If you see several exited containers from the `claude-dev` image with random names, that’s the tell. Keep the one you’ve logged into, rename it for clarity, and remove the rest:

```
docker rename <random_name> claude-dev-main
docker rm <other_stale_container_names>
```

5. Authenticate

The first `claude` run opens a browser flow. If the container has no browser, the terminal shows a “paste code here” prompt — copy the code from the browser on your host and paste it in.

For fully headless or CI use, run `claude setup -token` somewhere interactive and pass the result in as `CLAUDE_CODE_OAUTH_TOKEN`, or supply `ANTHROPIC_API_KEY`.

Claude Code requires a Pro, Max, Team, Enterprise, or Console account. The free plan doesn't include access.

Confirm login actually persisted

Right after a successful login, verify the credential file exists and is owned correctly:

```
ls -la ~/.claude/
whoami
ls -ld ~/.claude
```

You want to see files inside owned by `dev:dev`, not `root:root`. If the Dockerfile fix in step 3 was applied, this should just work; the check is worth doing once to be sure, especially on a freshly built image.

Troubleshooting: “login successful”, then “not logged in”

This specific pattern — the CLI accepts the login, then on the very next check reports you're logged out — has two root causes in practice. Both are addressed by the changes in this revision, but if you're debugging an existing image built from the old recipe, here's how to tell them apart.

Cause	How to confirm	Fix
<code>/home/dev/.claude</code> is owned by root	<code>ls -la /home/dev/.claude</code> shows <code>root root</code> on the directory itself, even though <code>/home/dev</code> is owned by <code>dev</code>	<code>chown -R dev:dev /home/dev/.claude</code> (or rebuild the image with the fix in step 3)
A new container was created instead of reusing the logged-in one	<code>docker ps -a</code> shows multiple containers from the <code>claude-dev</code> image with random names; the one you're currently in isn't the one you logged into	Always reconnect with <code>docker start -ai claude-dev-main</code> or <code>docker exec -it claude-dev-main bash</code> . Never repeat <code>docker run</code> for daily use.
Container was started with <code>--rm</code> and has since exited	The container is simply gone from <code>docker ps -a</code> — not stopped, not listed at all	Drop <code>--rm</code> from the run command (see step 4). Rely on the named volume for durability instead of keeping the container disposable.
<code>ANTHROPIC_API_KEY</code> set alongside interactive login	<code>env grep -i anthropic</code> shows a key is set	<code>unset ANTHROPIC_API_KEY</code> before running <code>claude login</code> , or decide on one auth method and stick with it

Quickest single diagnostic: run `docker ps -a` before assuming anything is broken. Most “auth keeps disappearing” reports turn out to be several different containers, not one flaky one.

6. Start the container without launching Claude

The image ends with CMD ["c laude"], which is the only reason the CLI comes up on its own. Anything you type after the image name in `docker run` replaces that command. That one detail is all it takes to get a shell first, install whatever else the container needs, and start Claude by hand afterward.

First run — a shell instead of Claude

Identical to the run command in step 4, with `bash` appended:

```
docker run -it --name claude-dev-main \
  -v "$PWD":/workspace \
  -v claude-config:/home/dev/.claude \
  -e CLAUDE_CONFIG_DIR=/home/dev/.claude \
  claude-dev bash
```

You land as `dev` in `/workspace`. The Dockerfile in step 3 gave `dev` passwordless `sudo`, so installing is unremarkable:

```
sudo apt-get update
sudo apt-get install -y build-essential gdb cmake python3-pip
```

Type `c laude` when the container has what you want in it. Exiting the shell stops the container, but does not delete it — `--rm` was already dropped in step 4, so it is still there in `docker ps -a` with everything you installed intact.

A container that already exists

Do not reach for `docker start -ai` here. It re-runs whatever command the container was created with, and for a container made the step 4 way that command is `c laude`. Start it detached and attach a shell separately:

```
# Boot it without attaching to its original command
docker start claude-dev-main
```

```
# Get a shell -- CMD is not involved at all
docker exec -it claude-dev-main bash
```

```
# Same thing as root, if you would rather skip sudo
docker exec -it -u root claude-dev-main bash
```

`docker exec` never runs the image's CMD, so this works whether Claude is running in the container or not — you can install a package in one shell while a Claude session is live in another.

Command	What actually runs
<code>docker run ... claude-dev</code>	The image CMD — Claude starts immediately.
<code>docker run ... claude-dev bash</code>	A shell. The CMD is replaced for the life of that container, so a later <code>docker start -ai</code> on it gives a shell too.
<code>docker start -ai claude-dev-main</code>	Whatever command the container was originally created with. Attaches you to it.

Command	What actually runs
<pre>docker start ... docker exec -it ... bash</pre>	A shell alongside whatever the container is already running. The everyday way in.
<pre>docker exec -it -u root ... bash</pre>	A root shell in the running container. Handy when sudo is missing or apt is unhappy.

Where those packages actually live

Interactive installs are not part of the image. Packages installed with `apt-get` inside a running container land in that container's writable layer. They survive stop and start indefinitely, but they are gone the moment you `docker rm` the container or rebuild the image. Neither the `/workspace` bind mount nor the `claude-config` volume covers them — those hold your project files and your login, not installed software.

Making it permanent — move it into the Dockerfile

Once you know which packages you actually need, add them to the Dockerfile from step 3. The `apt` install has to sit **above** the `USER dev` line, since `apt` needs root:

```
# --- Extra tooling ---
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential gdb cmake python3-pip \
    && rm -rf /var/lib/apt/lists/*
```

Then rebuild and recreate the container:

```
docker build -t claude-dev .
docker rm claude-dev-main
docker run -it --name claude-dev-main \
    -v "$PWD":/workspace \
    -v claude-config:/home/dev/.claude \
    -e CLAUDE_CONFIG_DIR=/home/dev/.claude \
    claude-dev
```

Layer caching means only the new `RUN` line and everything below it is re-executed, so this is fast after the first time. The login survives because the credentials are in the `claude-config` named volume, not in the container being replaced — which is the whole reason step 4 mounts one.

Worked example — adding gcc and git

Two things before editing anything. `git` is already installed — the first `RUN` in step 3 lists `ca-certificates` `curl` `gnupg` `git` `ripgrep` `sudo`, so the only real addition is a compiler. And `gcc` on its own gives you the compiler and the `libc` headers but no `make` and no `g++`. `build-essential` is what `Makefiles` and `./configure` scripts actually assume is present — `gcc`, `g++`, `make`, `libc6-dev` and `dpkg-dev`, around 200 MB. So rather than a new layer, extend the package list that is already there:

```
# The first RUN from step 3, with a toolchain added
RUN apt-get update && apt-get install -y --no-install-recommends \
    ca-certificates curl gnupg git ripgrep sudo \
    build-essential gdb \
    && rm -rf /var/lib/apt/lists/*
```

Nothing else in the `Dockerfile` changes. Four details decide whether this behaves:

- **Keep it in that first RUN** — above the Claude Code repo block, and well above `USER dev`. `Apt` needs root, and `USER dev` is the line after which it does not have it.
- **Editing that line invalidates the cache below it.** The Claude Code install re-runs as well. That is a one-time cost of a couple of minutes, not a reason to avoid the edit.
- **Leave `--no-install-recommends` on.** Without it, `git` alone pulls in documentation packages and `build-essential` gets noticeably fatter.
- **The `rm -rf /var/lib/apt/lists/*` has to stay in the same RUN.** In a separate one it deletes nothing from the image — the earlier layer still carries the package index, and the image keeps paying for it.

Rebuild with the same three commands as above. For a microcontroller toolchain, the same line is where `gcc-arm-none-eabi`, `avrdude`, `openocd` or a vendor SDK would go — with the caveat that a USB programmer needs `--device` passed to `docker run`, which is a step 4 change rather than a `Dockerfile` one.

The workflow this suggests: use a throwaway shell to work out what the container is missing, then move the finished package list into the `Dockerfile` and rebuild. The interactive install is for figuring out the list; the `Dockerfile` is for keeping it. Toolchains are far too much apt to re-type by hand every time the image changes.

If Claude should never autostart

If the container is really a development box that sometimes runs Claude, rather than the other way around, change the last line of the `Dockerfile`:

```
CMD ["bash"]
```

After that, `docker run` and `docker start -ai` both drop you at a prompt, and Claude is just a command you type. Nothing else in the recipe changes — the ownership fix, the config directory, and the named volume all still do their jobs.

Dev container alternative

If you want VS Code attached to it rather than a bare terminal, skip the manual install — there's an official dev container feature. The same two fixes apply here: a named volume for the config directory, and letting VS Code manage the single container instead of recreating it by hand.

```
{
  "image": "mcr.microsoft.com/devcontainers/base:ubuntu",
  "features": {
    "ghcr.io/anthropics/devcontainer-features/claude-code:1.0": {}
  },
  "mounts": ["source=claude-code-config,target=/home/vscode/.claude,type=volume"],
  "containerEnv": { "CLAUDE_CONFIG_DIR": "/home/vscode/.claude" }
}
```

The reference `.devcontainer/` in the claude-code repo is a working example that adds an egress firewall (`init-firewall.sh`) restricting outbound traffic to only the domains Claude Code needs. Worth copying if you plan to run with `--dangerously-skip-permissions`, since the container is otherwise free to reach anything on the network.

With VS Code's Dev Containers extension, “Reopen in Container” and “Rebuild Container” manage the container lifecycle for you, so the manual “which random container am I in” problem mostly doesn't arise here — the named volume is still what makes rebuilds keep your login.

Things that tend to bite

- **uid collision.** `ubuntu:24.04` ships with a default `ubuntu` user at uid 1000, so the `useradd dev` above gets uid 1001. If bind-mount file ownership matters, use the existing `ubuntu` user instead, or pass `--uid` explicitly.
- **Root-owned config directory.** The single most common cause of “login successful” not sticking. Fixed by creating and chowning `/home/dev/.claude` before `USER dev` in the Dockerfile (see step 3). If you inherit an older image without this line, `chown -R dev:dev /home/dev/.claude` fixes it in place.
- **Container sprawl.** Repeated bare `docker run` calls create a new, unauthenticated container each time. Name the container once, reconnect with `docker start -ai` or `docker exec` afterward, and periodically clean up stray containers with `docker rm`.
- **Group membership.** After `usermod -aG docker`, a fresh login is required — `newgrp` alone only affects the current shell.
- **Network.** The container needs outbound internet to reach Anthropic's API. Confirm that before debugging a hang on first launch.
- **Resources.** Claude Code wants 4 GB+ RAM. Both x86-64 and ARM64 hosts are supported.
- **Rebuilds are cheap.** Docker caches each layer, so editing the Dockerfile and re-running `docker build` only re-executes from the changed line down. Don't be afraid to iterate on step 3.

Sources

- Docker Docs — Install Docker Engine on Ubuntu
- Docker Docs — Linux post-installation steps
- Claude Code — Advanced setup (apt repository and key fingerprint)
- Claude Code — Development containers